

Instituto Tecnológico de Costa Rica

Campus Tecnológico Local San José

Laboratorio 03: Consultas

Profesor:

Adriana Álvarez Figueroa

Estudiantes:

González Prendas Mauricio: 2024143009

Hidalgo Paz Carmen: 2020030538

Meza Marin Dylan: 2021577352

Fecha de entrega:

21/04/25

Semestre I 2025

1. Consulte la cantidad total de personas que existen. 5 pts.

Para lograr esto se utilizó la función Count(1).

La tabla:

	ID	FIRST_NAME	SECOND_NAME	FIRST_SURNAME	SECOND_SURNAME	SALARY	BIRTHDAY	ID_TYPE_PEOPLE
1	2	Marcela	Luisa	Martínez	López	19837.5	15-MAR-78	1
2	3	Pedro	Alberto	Sánchez	Ramírez	21160	20-JUL-85	1
3	4	Ana	Isabel	Hernández	Morales	21160	05-NOV-82	1
4	5	Luis	Fernando	García	Torres	22482.5	28-FEB-79	2
5	6	Sofía	Elena	Ruiz	Castro	22482.5	12-SEP-83	2
6	7	Miguel	Angel	Díaz	Vargas	23805	03-APR-81	2
7	8	Laura	Milagros	Moreno	Ramos	23805	17-AUG-84	2
8	9	Carlos	Andrés	Ortiz	Silva	25127.5	22-DEC-77	2
9	10	Elena	Beatriz	Mendoza	Cruz	25127.5	30-JUN-86	2
10	12	Patricia	Soledad	Vega	Delgado	2777.25	22-MAR-92	2
11	13	Jorge	Andrés	Castro	Muñoz	2909.5	07-JUL-88	2
12	14	Cecilia	Marina	Romero	Paredes	3041.75	18-SEP-91	2
13	15	Ricardo	Fabián	Guerrero	Cabrera	3174	11-NOV-87	2
14	16	Verónica	Diana	Aguilar	Soto	3306.25	25-APR-93	2
15	17	Andrés	Manuel	Rojas	Campos	3438.5	05-JUN-89	2
16	18	Gloria	Estela	Molina	Fuentes	3570.75	01-DEC-90	2
17	19	Raúl	Emilio	Salinas	Quintero	3703	10-OCT-88	2
18	20	Natalia	Rosa	Cifuentes	Valdez	3835.25	05-MAY-92	2

El código:

```
---
---Descripción:
--- Se utiliza la función COUNT para obtener la cantidad total de
--- personas en la tabla ge.people
---
SELECT COUNT(1) AS total_personas
FROM ge.people;
```

El resultado:

Script Output x Query Result x	
All Rows Fetched: 1 in 0.007 seconds	
TOTAL_PERSONAS	
1	18

2. Haga un sql para obtener todas las personas cuyo nombre empiece con la letra B. 4 pts.

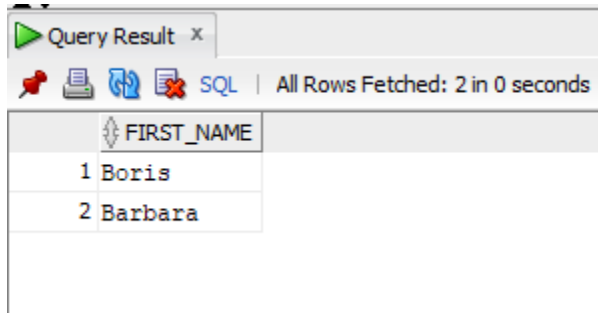
Para este ejercicio se agregaron tres personas más a la tabla. Se utiliza el operador LIKE para buscar la primera letra de los nombres. Ahora se ve así:

	ID	FIRST_NAME	SECOND_NAME	FIRST_SURNAME	SECOND_SURNAME	SALARY	BIRTHDAY	ID_TYPE_PEOPLE
1	21	Boris	Adrián	Campos	Hernandez	7600	02-APR-76	1
2	22	Barbara	María	Solís	Vargas	2600	02-AUG-82	2
3	23	bianca	paola	mora	cascante	3400	24-DEC-99	1
4	2	Marcela	Luisa	Martínez	López	19837.5	15-MAR-78	1
5	3	Pedro	Alberto	Sánchez	Ramírez	21160	20-JUL-85	1
6	4	Ana	Isabel	Hernández	Morales	21160	05-NOV-82	1
7	5	Luis	Fernando	García	Torres	22482.5	28-FEB-79	2
8	6	Sofía	Elena	Ruiz	Castro	22482.5	12-SEP-83	2
9	7	Miguel	Angel	Díaz	Vargas	23805	03-APR-81	2
10	8	Laura	Milagros	Moreno	Ramos	23805	17-AUG-84	2
11	9	Carlos	Andrés	Ortiz	Silva	25127.5	22-DEC-77	2
12	10	Elena	Beatriz	Mendoza	Cruz	25127.5	30-JUN-86	2
13	12	Patricia	SoledGE	Vega	DelgGEo	2777.25	22-MAR-92	2
14	13	Jorge	Andrés	Castro	Muñoz	2909.5	07-JUL-88	2
15	14	Cecilia	Marina	Romero	Paredes	3041.75	18-SEP-91	2
16	15	Ricardo	Fabián	Guerrero	Cabrera	3174	11-NOV-87	2
17	16	Verónica	Diana	Aguilar	Soto	3306.25	25-APR-93	2
18	17	Andrés	Manuel	Rojas	Campos	3438.5	05-JUN-89	2
19	18	Gloria	Estela	Molina	Fuentes	3570.75	01-DEC-90	2
20	19	Raúl	Emilio	Salinas	Quintero	3703	10-OCT-88	2
21	20	Natalia	Rosa	Cifuentes	Valdez	3835.25	05-MAY-92	2

El código:

```
---
---Descripción:
--- Se utiliza el operador LIKE para obtener los primeros nombres que
--- empiecen con B. El % es para decir que no importa la cantidad
--- de caracteres que le siguen.
---
SELECT first_name
FROM ge.people
WHERE first_name LIKE 'B%';
```

El resultado:



Query Result x

SQL | All Rows Fetched: 2 in 0 seconds

	FIRST_NAME
1	Boris
2	Barbara

3. Haga un sql para obtener todas las personas cuyo nombre empiece con la letra b (en minúscula). 1 pts.

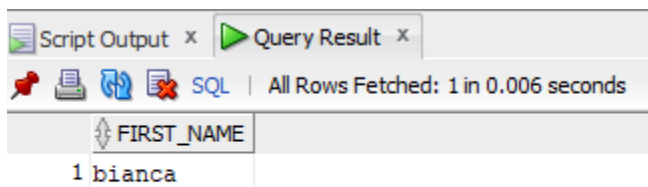
Se utiliza la tabla de personas modificada en el ejercicio 2. El operador LIKE de Oracle distingue entre mayúsculas y minúsculas. Por lo tanto, el código es muy parecido al ejercicio anterior.

El código:

```
---
---Descripción:
--- Se utiliza el operador LIKE para obtener los primeros nombres que
--- empiecen con b. El % es para decir que no importa la cantidad
--- de caracteres que le siguen.
---

SELECT first_name
FROM ge.people
WHERE first_name LIKE 'b%';
```

El resultado:



Script Output x Query Result x

SQL | All Rows Fetched: 1 in 0.006 seconds

	FIRST_NAME
1	bianca

4. Indique la cantidad total de números de teléfono por persona. 10 pts.

Para obtener también las personas que tienen 0 teléfonos se hizo un LEFT JOIN. Esta es la tabla de PHONE:

	ID	PHONE_NUMBER	ID_TYPE_PHONE
1	1	555-1234	1
2	2	555-5678	2
3	3	555-9012	3
4	4	555-3456	1
5	5	555-7890	2
6	6	555-2345	3
7	8	555-0123	1

Esta es la tabla de PHONEXPEOPLE:

	ID_PEOPLE	ID_PHONE
1	5	1
2	5	3
3	5	6
4	10	8
5	2	6
6	10	1
7	2	2
8	3	3
9	4	4
10	5	5
11	6	6
12	7	1
13	8	8
14	9	3
15	10	4
16	12	6
17	13	1
18	14	2
19	15	3
20	16	5
21	17	4
22	18	8
23	19	6
24	20	1

Este es el código del cálculo:

```
---
---Descripción:
--- Se obtiene la cantidad total de números por persona.
---
SELECT p.id, p.first_name, COUNT(pp.id_phone) AS phone_count
FROM ge.people p
LEFT JOIN ge.phonexpeople pp ON p.id = pp.id_people
GROUP BY p.id, p.first_name
ORDER BY p.id;
```

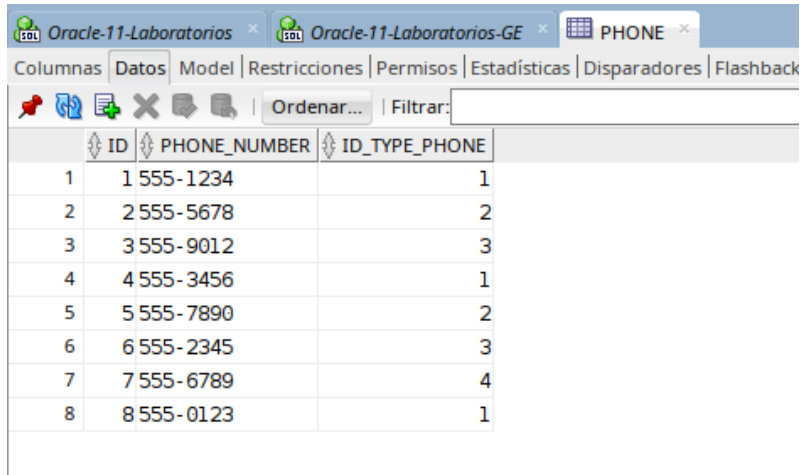
Este es el resultado:

Query Result x			
All Rows Fetched: 21 in 0.011 seconds			
ID	FIRST_NAME	PHONE_COUNT	
1	2 Marcela	2	
2	3 Pedro	1	
3	4 Ana	1	
4	5 Luis	4	
5	6 Sofía	1	
6	7 Miguel	1	
7	8 Laura	1	
8	9 Carlos	1	
9	10 Elena	3	
10	12 Patricia	1	
11	13 Jorge	1	
12	14 Cecilia	1	
13	15 Ricardo	1	
14	16 Verónica	1	
15	17 Andrés	1	
16	18 Gloria	1	
17	19 Raúl	1	
18	20 Natalia	1	
19	21 Boris	0	
20	22 Barbara	0	
21	23 bianca	0	

5. Haga un sql que retorne todas las personas que tengan teléfono tipo ‘Casa’. 10 pts.
- Para obtener todas las personas que tengan al menos un teléfono de tipo “Casa” (tipo identificado con los IDs 1, 4 y 8, siendo el ID 1 el correspondiente a “Casa”), se analiza la tabla PHONEXPEOPLE y se observa que hay 12 registros asociados a ese tipo; sin embargo, la persona con ID 10 aparece tres veces, por lo que al emplear SELECT

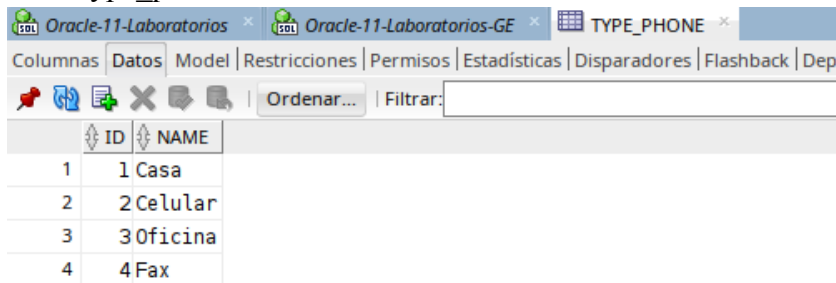
DISTINCT junto con los INNER JOIN correspondientes se garantiza que cada persona se liste una sola vez en el resultado.

Tabla Phone



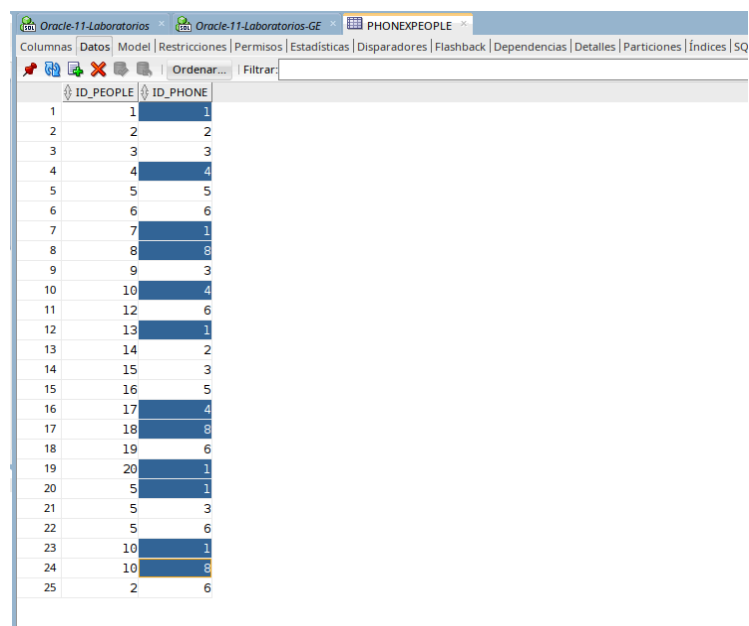
ID	PHONE_NUMBER	ID_TYPE_PHONE
1	1 555-1234	1
2	2 555-5678	2
3	3 555-9012	3
4	4 555-3456	1
5	5 555-7890	2
6	6 555-2345	3
7	7 555-6789	4
8	8 555-0123	1

Tabla type_phone



ID	NAME
1	1 Casa
2	2 Celular
3	3 Oficina
4	4 Fax

Tabla PhoneXPeople



ID_PEOPLE	ID_PHONE
1	1
2	2
3	3
4	4
5	5
6	6
7	1
8	6
9	3
10	4
11	6
12	1
13	2
14	3
15	5
16	4
17	6
18	6
19	1
20	1
21	3
22	6
23	1
24	6
25	6

El resultado:

ID	FIRST_NAME	SECOND_NAME	FIRST_SURNAME	SECOND_SURNAME	SALARY	BIRTHDAY
13	Jorge	Andrés	Castro	Muñoz	2200	07/07/8
5	Luis	Fernando	García	Torres	17000	28/02/7
18	Gloria	Estela	Molina	Fuentes	2700	01/12/9
17	Andrés	Manuel	Rojas	Campos	2600	05/06/8
20	Natalia	Rosa	Cifuentes	Valdez	2900	05/05/9
7	Miguel	Angel	Díaz	Vargas	18000	03/04/8
10	Elena	Beatriz	Mendoza	Cruz	19000	30/06/8
8	Laura	Milagros	Moreno	Ramos	18000	17/08/8
1	Juan	Carlos	Pérez	Gómez	15000	10/05/8
4	Ana	Isabel	Hernández	Morales	16000	05/11/8

10 filas seleccionadas.

6. Cree una vista que contenga a todas las personas que ganan menos de \$3000. 10 pts.

Codigo:

```
CREATE OR REPLACE VIEW GE.V_LOW_SALARY_PEOPLE AS
SELECT id, first_name, first_surname, salary
FROM GE.PEOPLE
WHERE salary < 3000;
```

Resultado:

ID	FIRST_NAME	FIRST_SURNAME	SALARY
1	11 Diego	Flores	2000
2	12 Patricia	Vega	2100
3	13 Jorge	Castro	2200
4	14 Cecilia	Romero	2300
5	15 Ricardo	Guerrero	2400
6	16 Verónica	Aguilar	2500
7	17 Andrés	Rojas	2600
8	18 Gloria	Molina	2700
9	19 Raúl	Salinas	2800
10	20 Natalia	Cifuentes	2900
11	22 Barbara	Solís	2600

7. Cree dos vistas con el top 3 de personas con más salario con base en los 2 siguientes queries. ¿Cuál es la diferencia entre cada uno? 20 pts.

Codigo:

```
CREATE OR REPLACE VIEW GE.V_TOP3_SALARY_ROWNUM AS
SELECT ROWNUM AS id,
       first_name || ' ' || first_surname AS nombre,
       salary
FROM (SELECT first_name, first_surname, salary
      FROM GE.PEOPLE
      ORDER BY salary DESC)
WHERE ROWNUM <= 3;

-- Vista 2 (RANK)
CREATE OR REPLACE VIEW GE.V_TOP3_SALARY_RANK AS
SELECT first_name || ' ' || first_surname AS nombre,
       salary,
       RANK() OVER (ORDER BY salary DESC) salary_rank
FROM GE.PEOPLE
WHERE salary_rank <= 3;

/*
Diferencia:
- ROWNUM: Estricto top 3 físico, ignora empates
- RANK: Incluye todos los registros con el mismo rango, podría devolver más de 3 si hay empates
*/
```

Resultado con rownum, se puede observar que solo muestra 3 porque ignora los empates:

Oracle-11-Laboratorios			
Oracle-11-Laboratorios-GE			
V_TOP3_SALARY_ROWNUM			
Columnas Datos Permisos Dependencias Detalles Disparadores SQL Errores			
Ordenar... Filtrar:			
ID	NOMBRE	SALARY	
1	1 Carlos Ortiz	19000	
2	2 Elena Mendoza	19000	
3	3 Miguel Díaz	18000	

Resultado con rank, se puede observar que puede superar los 3 items si hay empates.

Oracle-11-Laboratorios			
Oracle-11-Laboratorios-GE			
V_TOP3_SALARY_RANK			
Columnas Datos Permisos Dependencias Detalles Disparadores SQL Errores			
Ordenar... Filtrar:			
	NOMBRE	SALARY	
1	Carlos Ortiz	19000	
2	Elena Mendoza	19000	
3	Miguel Díaz	18000	
4	Laura Moreno	18000	

8. Consulte la cantidad total de clientes que tienen compras. 5 pts.

Codigo:

```
Hoja de Trabajo | Generador de Consultas

-- 08.sql - Conteo de Clientes que han realizado compras
--
-- Descripción: Esta consulta calcula el número total de clientes únicos
-- que han realizado al menos una compra en el sistema. Utiliza DISTINCT
-- para asegurar que cada cliente se cuente una sola vez, independientemente
-- del número de compras que haya realizado.
--
-- Tablas utilizadas:
-- - PEOPLE: Información de las personas (incluye clientes)
-- - BUY: Registro de compras realizadas por los clientes
--
-- Filtros:
-- - id_type_people = 2: Asegura que solo se consideren personas de tipo Cliente
--
-- Resultado:
-- - total_clientes_compras: Cantidad total de clientes distintos que han realizado compras
SELECT COUNT(DISTINCT p.id) AS total_clientes_compras
FROM GE.PEOPLE p
INNER JOIN GE.BUY b ON p.id = b.id_people
WHERE p.id_type_people = 2; -- Tipo Cliente
```

Resultado:

```
Salida de Script x
Tarea terminada en 0,233 segundos

TOTAL_CLIENTES_COMPRAS
-----
6
```

9. Consulte la cantidad total de clientes que existen. 5 pts.

Para obtener este resultado se hizo un SELECT con un COUNT de la tabla GE.PEOPLE donde se contaban solo los que tenían el atributo id_type_people como cliente. Esta es la tabla de TYPE_PEOPLE:

	ID	NAME
1	1	Employee
2	2	Client

El código:

```

---
---Descripción:
--- Se utiliza la función COUNT para obtener la cantidad total de
--- clientes en la tabla ge.people
---
SELECT COUNT(1) AS total_clientes
FROM ge.people p
WHERE p.id_type_people = 2;

```

El resultado:

	TOTAL_CLIENTES
1	16

10. Liste los productos comprados por un cliente agrupado por cliente. 10 pts.

Esta sentencia une PEOPLE, BUY, CART, PRODUCTXCART y PRODUCT para rastrear las compras, filtra solo clientes y agrupa por su ID. Luego, mediante una sub-consulta DISTINCT y LISTAGG, concatena en una sola cadena los nombres de los productos distintos que cada cliente ha adquirido, entregando por fila el historial resumido de compras de cada cliente.

Codigo:

```

-- - productos_comprados. Lista consolidada de todos los productos adquiridos
SELECT
  datos.id_cliente,
  datos.nombre,
  LISTAGG(datos.producto, ', ')
    WITHIN GROUP (ORDER BY datos.producto) AS productos_comprados
FROM (
  /* --- sub-consulta que quita duplicados --- */
  SELECT DISTINCT
    p.id AS id_cliente,
    p.first_name || ' ' || p.first_surname AS nombre,
    pr.name AS producto
  FROM GE.PEOPLE p
  JOIN GE.BUY b ON b.id_people = p.id
  JOIN GE.CART c ON c.id = b.id_cart
  JOIN GE.PRODUCTXCART pc ON pc.id_cart = c.id
  JOIN GE.PRODUCT pr ON pr.id = pc.id_product
  WHERE p.id_type_people = 2 -- solo clientes
) datos
GROUP BY
  datos.id_cliente,
  datos.nombre
ORDER BY
  datos.id_cliente;

```

El resultado:

Salida de Script x	
Tarea terminada en 0,285 segundos	
ID_CLIENTE	NOMBRE

PRODUCTOS_COMPRADOS	

6	Sofía Ruiz
Aceite, Detergente, Harina, Pasta, Pollo, Queso, Tomate	
7	Miguel Díaz
Azucar, Cereal, Desodorante, Detergente, Lechuga, Pollo, Shampoo	
8	Laura Moreno
Aceite, Cereal, Leche, Lechuga, Natilla, Pasta, Pescado, Queso, Shampoo	
ID_CLIENTE	NOMBRE

PRODUCTOS_COMPRADOS	

9	Carlos Ortiz
Desodorante, Detergente, Harina, Pescado, Pollo, Queso	
10	Elena Mendoza
Leche, Pasta	
12	Patricia Vega
Aceite, Pollo	

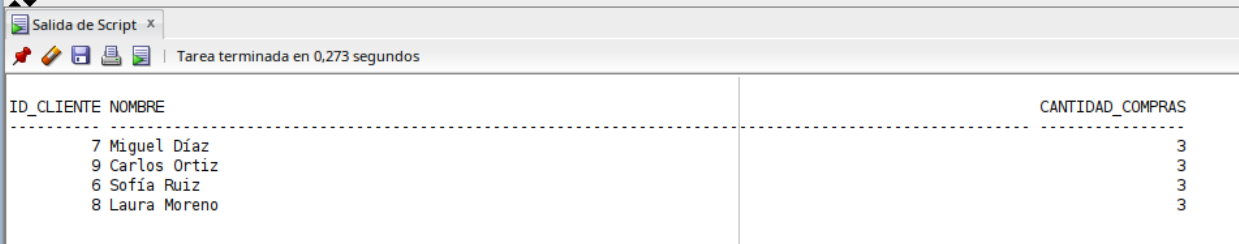
11. Haga un sql de todos los clientes que tienen más de 2 compras. 20 pts.

Esta consulta agrupa las compras por cliente, cuenta cuántas ha realizado cada uno y, tras filtrar solo a los clientes (id_type_people = 2) con más de dos transacciones, devuelve su ID, nombre completo y total de compras, ordenados de mayor a menor para resaltar a los compradores más frecuentes.

El código:

```
-- - cantidad_compras: total de transacciones realizadas por el cliente
SELECT
  p.id AS id_cliente,
  p.first_name || ' ' || p.first_surname AS nombre,
  COUNT(*) AS cantidad_compras
FROM   GE.PEOPLE p
JOIN   GE.BUY b ON b.id_people = p.id
WHERE  p.id_type_people = 2 -- solo clientes
GROUP BY p.id, p.first_name, p.first_surname
HAVING COUNT(*) > 2 -- más de dos compras
ORDER BY cantidad_compras DESC;
```

El resultado:



Salida de Script x

Tarea terminada en 0,273 segundos

ID_CLIENTE	NOMBRE	CANTIDAD_COMPRAS
7	Miguel Díaz	3
9	Carlos Ortiz	3
6	Sofía Ruiz	3
8	Laura Moreno	3